

## Ethereum accounts:

Accounts are the *main building blocks* for the Ethereum ecosystem.

It is an interaction between accounts that Ethereum wants to store as transactions in its ledger.

There are two types of accounts available in Ethereum—**externally owned accounts** and **contract accounts**.

Each account, by default, has a property named balance that **helps in querying the current balance of Ether**.

\*\*\*\*\*

### Externally owned accounts

Externally owned accounts are *accounts that are owned by people on Ethereum*. Accounts are not referred to by name in Ethereum.

When an externally owned account is created on Ethereum by an individual, **a public/private key is generated**.

The private key is kept safe with the individual while the public key becomes the identity of this externally owned account.

This public key is generally of **256 characters**, however, Ethereum uses the **first 160 characters** to represent the identity of an account.

\*\*\*\*\*

An externally owned account *can hold Ether in its balance* and *does not have any code associated with it*. It can *execute transactions with other externally owned accounts* and it can also *execute transactions by invoking functions within contracts*.

\*\*\*\*\*

## Contract accounts:

Contract accounts are **very similar to externally owned accounts**. They are identified using their public address. **They do not have a private key**. They can **hold Ether** similar to externally owned accounts; however, **they contain code**—**code for smart contracts consisting of functions** and **state variables**.

\*\*\*\*\*

Following are the *three types of transactions* that can be executed in Ethereum:

- **Transfer of Ether from one account to another**: The accounts can be externally owned accounts or contract accounts.
- **Deployment of a smart contract**: An externally owned account can deploy a contract using a transaction in EVM.
- **Using or invoking a function within a contract**: Executing a function in a contract that changes state is considered a transaction in Ethereum. If executing a function does not change a state, it does not require a transaction.

\*\*\*\*\*

The **to account property** refers to an account that is receiving Ether or benefits in lieu of an exchange. For transactions related to deployment of contract, the to field is empty. It can be externally owned or a contract account.

\*\*\*\*\*

The **input account property** refers to the compiled contract bytecode and **is used during contract deployment in EVM**. It is also **used for storing data related to smart contract** function calls along with its parameters. A typical transaction in Ethereum where a contract function is invoked is shown here.

\*\*\*\*\*

The **v, r, and s account** properties relate to *digital signatures* and the *signing of the transaction*.

\*\*\*\*\*

**Blocks** are **containers** for a **transaction**. A block **contains multiple transactions**. Each block has a **different number** of **transactions** based on **gas limit** and **block size**.

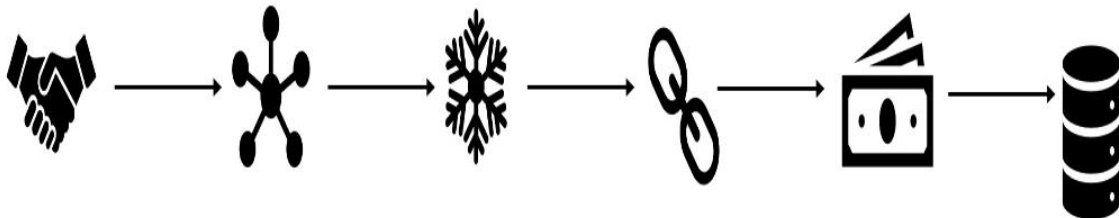
\*\*\*\*\*

The *gasLimit* property *determines the maximum gas allowed*. This helps in **determining** how many **transactions** can be **part of the block**.

\*\*\*\*\*

The **receiptsRoot**, **stateRoot**, and **transactionsRoot** properties refer to Merkle trees.

\*\*\*\*\*



*Sam wants  
to send \$\$  
to Mark*

*Transaction  
reaches the  
network*

*The nodes  
compete  
and  
validate  
the  
transaction*

*A new block  
with  
transaction  
is  
generated*

*Both  
Accounts  
are  
updated*

*The Block is  
replicated  
and stored  
on all nodes*

\*\*\*\*\*

## What is a contract?

A contract is a legal document that binds two or more parties who agree to execute a transaction immediately or in the future. Since contracts are legal documents, they are enforced and implemented by law.

\*\*\*\*\*

## What is a smart contract?

A smart contract is custom logic and code deployed and executed within an Ethereum virtual environment.

Smart contracts are digitized and codified rules of transaction between accounts.

Smart contracts help in transferring digital assets between accounts as an atomic transaction.

Smart contracts can store data. The data stored can be used to record information, facts, associations, balances, and any other information needed to implement logic for real-world contracts.

Smart contracts are very similar to object-oriented classes. A smart contract can *call another smart contract* just like an object-oriented object can create and use objects of another class.

Think of smart contracts as a small program consisting of functions. You can create an instance of the contract and invoke functions to view and update contract data along with the execution of some logic.

\*\*\*\*\*

## How to write smart contracts?

There are multiple smart contracts authoring tools including Visual Studio. However, the easiest and fastest way to develop smart contracts is to use a *browser-based tool* known as Remix. Remix is available on <http://remix.ethereum.org>. Remix is a new name and was earlier known as *browser-solidity*.

All contract management related activities such as *authoring*, *deploying*, and *troubleshooting* can be performed from the same environment without moving to other windows or tabs.

\*\*\*\*\*

Not everyone is comfortable using the online version of Remix to author their smart contracts. Remix is an open source tool that can be downloaded from <https://github.com/ethereum/browser-Solidity> and compiled to run a private version on a local computer.

\*\*\*\*\*

The contract is created using the contract keyword; you can declare global state variables and functions; and contracts are saved with the `.sol` file extension.

\*\*\*\*\*

```
// SPDX-License-Identifier: MIT
pragma solidity >=0.7.0 <0.9.0;

/**
 * @title HelloWorld
 * @dev HelloWorld
 * @custom:dev-run-script scripts/web3-lib.ts
 */
contract HelloWorld {
    string private stateVariable = "Hello Remix World";

    function GetHelloWorld() public view returns (string memory)
    {
        return stateVariable;
    }
}
```

\*\*\*\*\*

Note 1: While we call the state variable that can be modified, we must set *memory*

Note 2: With memory, we also have *calldata*, that are used for *readonly* data with *external-keyword*, and it's less gas consuming.

Note 3: the first comment must set From Solidity ^0.6.8

\*\*\*\*\*

The compiler generates the following two major artifacts:

- ABI definition
- Contracts bytecode

\*\*\*\*\*

Think of the *Application Binary Interface (ABI)* as an interface consisting of all external and public function declarations along with their parameters and return types. The ABI defines the contract and any caller wanting to invoke any contract function can use the ABI to do so.

\*\*\*\*\*

### Main network:

The main Ethereum network is a global public network that anybody can use. It can be accessed using an account and everybody is free to create an account and deploy their solutions and smart contracts. Using a main network incurs costs *in terms of gas*. The main network is known as Homestead and was earlier known as Frontier. This is a public chain accessible over the internet and anybody can connect to it and access both data and transactions stored in it.

\*\*\*\*\*

## Test network

A test network exists to help facilitate and increase adoption of the Ethereum blockchain. They are the exact replica of the main network. Using these networks does not cost anything for deployment and usage of contracts. They are completely free of cost. This is because **test Ethers** can be generated using faucets and used on these networks. There are multiple test networks available, such as Ropsten, Kovan, and Rinkeby.

\*\*\*\*\*

### Ropsten

Ropsten is one of the first test networks that use PoW consensus methods for generating blocks. It was earlier known as Morden. As mentioned before, it is completely free of cost for usage and it can be used during the building and testing of smart contracts. It can be used by using the *--testnet option available in Geth*.

\*\*\*\*\*

### Rinkeby

**Rinkeby** is another **Ethereum-based test network** that uses **PoA** as its consensus mechanism. **PoW** and **PoA** are different mechanisms for building consensus among miners. **PoW** is robust enough to maintain immutability and decentralization of data; however, it has drawbacks in not having enough control over miners. **PoA**, on the other hand, has all the benefits of **PoW** along with having more control over the miners.

\*\*\*\*\*

### Private network

A private network is created and hosted on *a private infrastructure*. *Private networks* are controlled by a single organization and they have full control over it. There are *solutions*, *contracts*, and *use cases* that an organization might not want to put on a public network even *for test purposes*. They want to use private chains for *development*, *testing*, and *production environments*. *Organizations* should create and host a private network and they will have full control over it.

\*\*\*\*\*

## Consortium network

A consortium network is also a private network, however, with a difference. The consortium network comprises nodes, each managed by a different organization. In effect, no organization has a control over the data and chain. However, it is shared within the organization and everyone from these organizations can view and modify the current state. These might be accessible **through the internet** or **completely private networks using VPN**.

\*\*\*\*\*

## Geth

Implementation of Ethereum nodes and clients is available in multiple languages, including **Go, C++, Python, JavaScript, Java, Ruby**, and more. The functionality or usability of these clients is the same across languages, and developers should choose an implementation they are most comfortable with. **Geth**, which acts as an Ethereum client to connect to public and test networks. It is also used to create the **mining and EVM (transaction nodes)** for private networks. **Geth** is a command-line tool written in Go for creating a node and miners on a private chain. It can be installed on Windows, Linux, and Mac as well.

\*\*\*\*\*

Geth is based on *JSON RPC protocol*. It defines the specification for remote procedure calls with payload encoded in JSON format. Geth allows *connectivity to JSON RPC* using the following three different protocols:  
*Inter Process Communication (IPC)*: This protocol is used for inter process communication generally used within the same computer.

*Remote Procedure Calls (RPC)*: This protocol is used for inter process communication across computers. This is generally based on TCP and HTTP protocol.

*Web Sockets (WS)*: This protocol is used to connect to Geth using sockets.

\*\*\*\*\*



*Geth* can be used to connect *to a public network* by just running *Geth* without any options. *Homestead* is the *current name of public Ethereum*. Its **networkid** and **ChainID** is 1.

\*\*\*\*\*

The following are the network IDs used for connecting to the following different networks:

- The chain ID 1 represents a Homestead public network
- The chain ID 2 represents Morden (not used anymore)
- The chain ID 3 represents Ropsten
- The chain ID 4 represents Rinkeby
- The chain ID above 4 represents a private network

\*\*\*\*\*

Geth provides the *--testnet* option to connect to a *Ropsten network* and the *--rinkeby option* to connect to the *Rinkeby option*. These should be used in conjunction with the networkid command option.

\*\*\*\*\*